

Experiment: Alpha-Beta Pruning Algorithm for Gaming Using Python

Aim

To implement the **Alpha-Beta Pruning algorithm** in Python for a two-player game and determine the optimal game value by eliminating branches that do not affect the final decision.

Objective

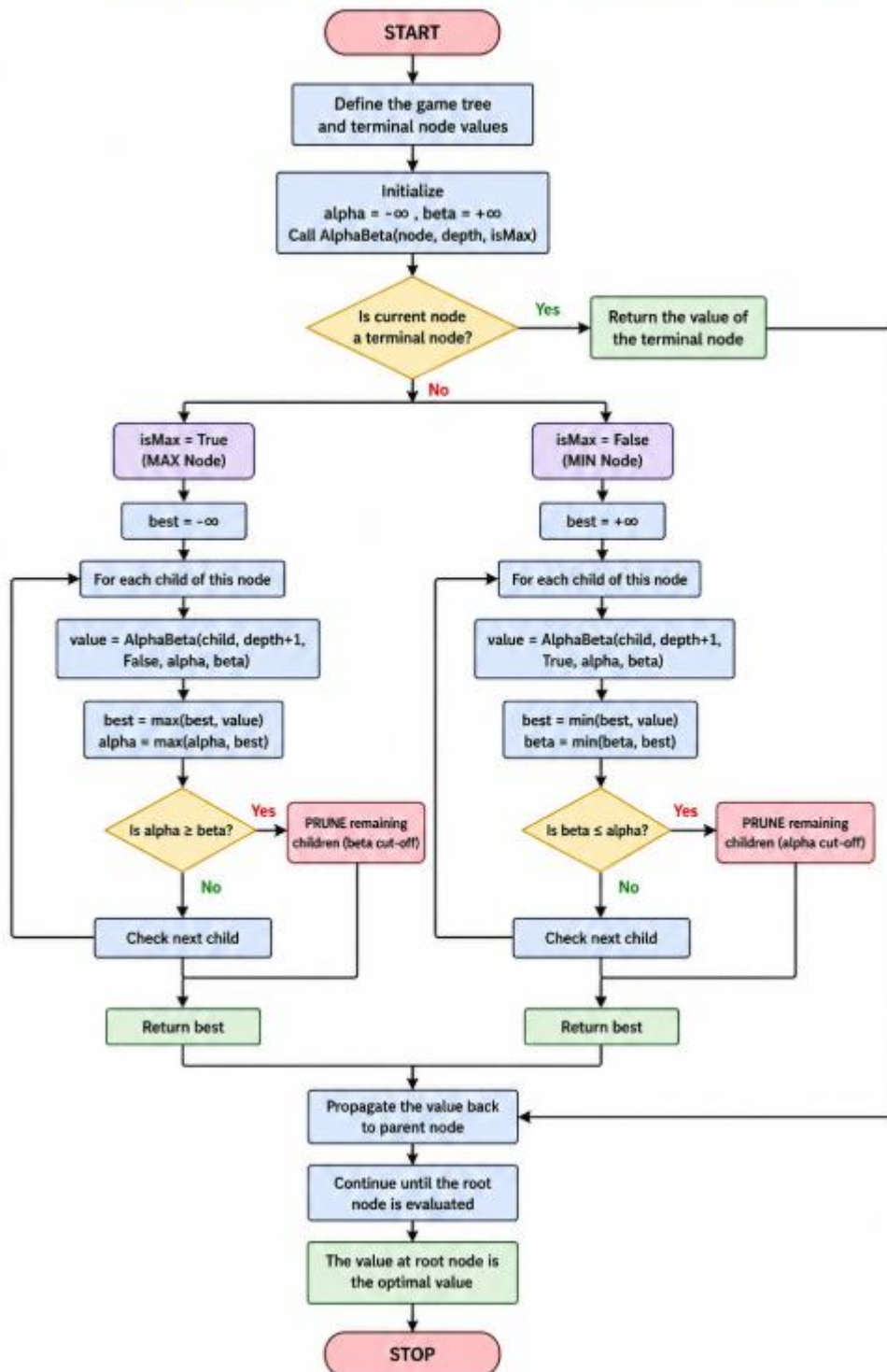
To implement Alpha-Beta Pruning as an optimization of the Minimax algorithm and reduce the number of game-tree nodes that need to be evaluated.

Algorithm

1. Start.
2. Define the game tree and terminal node values.
3. Initialize:
 - $\alpha = -\infty$
 - $\beta = +\infty$
4. Check whether the current node is a terminal node.
5. If it is a **MAX node**:
 - Evaluate its child nodes.
 - Update α .
 - If $\alpha \geq \beta$, prune the remaining branches.
6. If it is a **MIN node**:
 - Evaluate its child nodes.
 - Update β .
 - If $\beta \leq \alpha$, prune the remaining branches.
7. Continue until the root node is evaluated.
8. Display the optimal value.
9. Stop.

FLOWCHART:

FLOWCHART: ALPHA-BETA PRUNING ALGORITHM



Code:

```
import math
```

```
# Alpha-Beta Pruning Algorithm
```

```
def alpha_beta(depth, node, is_max, scores, height, alpha, beta):
```

```
    # Base condition: terminal node
```

```
    if depth == height:
```

```
        print("Terminal Node:", scores[node])
```

```
        return scores[node]
```

```
    if is_max:
```

```
        best = -math.inf
```

```
        for i in range(2):
```

```
            value = alpha_beta(
```

```
                depth + 1,
```

```
                node * 2 + i,
```

```
                False,
```

```
                scores,
```

```
                height,
```

```
                alpha,
```

beta

)

best = max(best, value)

alpha = max(alpha, best)

print(

"MAX Node:",

node,

"Value:",

best,

"Alpha:",

alpha,

"Beta:",

beta

)

Alpha-Beta pruning

if beta <= alpha:

print("PRUNED remaining branches at MAX Node:", node)

break

return best

else:

best = math.inf

for i in range(2):

value = alpha_beta(

depth + 1,

node * 2 + i,

True,

scores,

height,

alpha,

beta

)

best = min(best, value)

beta = min(beta, best)

print(

"MIN Node:",

node,

"Value:",

```
    best,  
    "Alpha:",  
    alpha,  
    "Beta:",  
    beta  
)
```

```
# Alpha-Beta pruning
```

```
if beta <= alpha:
```

```
    print("PRUNED remaining branches at MIN Node:", node)
```

```
    break
```

```
return best
```

```
# Terminal node values
```

```
scores = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
# Calculate tree height
```

```
height = int(math.log2(len(scores)))
```

```
# Initial alpha and beta
```

```
alpha = -math.inf
```

```
beta = math.inf
```

```
print("Alpha-Beta Pruning")
```

```
print("-----")
```

```
optimal_value = alpha_beta(
```

```
    0,
```

```
    0,
```

```
    True,
```

```
    scores,
```

```
    height,
```

```
    alpha,
```

```
    beta
```

```
)
```

```
print("\nOptimal Game Value:", optimal_value)
```

OUTPUT:

```
Alpha-Beta Pruning
-----
Terminal Node: 3
MAX Node: 0 Value: 3 Alpha: 3 Beta: inf
Terminal Node: 5
MAX Node: 0 Value: 5 Alpha: 5 Beta: inf
MIN Node: 0 Value: 5 Alpha: -inf Beta: 5
Terminal Node: 2
MAX Node: 1 Value: 2 Alpha: 2 Beta: 5
Terminal Node: 9
MAX Node: 1 Value: 9 Alpha: 9 Beta: 5
PRUNED remaining branches at MAX Node: 1
MIN Node: 0 Value: 5 Alpha: -inf Beta: 5
MAX Node: 0 Value: 5 Alpha: 5 Beta: inf
Terminal Node: 12
MAX Node: 2 Value: 12 Alpha: 12 Beta: inf
Terminal Node: 5
MAX Node: 2 Value: 12 Alpha: 12 Beta: inf
MIN Node: 1 Value: 12 Alpha: 5 Beta: 12
Terminal Node: 23
MAX Node: 3 Value: 23 Alpha: 23 Beta: 12
PRUNED remaining branches at MAX Node: 3
MIN Node: 1 Value: 12 Alpha: 5 Beta: 12
MAX Node: 0 Value: 12 Alpha: 12 Beta: inf

Optimal Game Value: 12
```

Result:

The **Alpha-Beta Pruning algorithm** was successfully implemented in Python. The algorithm determines the optimal game value while pruning unnecessary branches of the game tree.

Conclusion

Alpha-Beta Pruning improves the efficiency of the Minimax algorithm by eliminating branches that cannot affect the final decision. It is widely used in AI-based gaming applications such as **Tic-Tac-Toe, Chess, Checkers, and other two-player games.**

